

Requêtes HTTP avec POST

Nous avons vu comment envoyer des requêtes HTTP GET simple pour récupérer des données. Nous allons aller plus en détail sur le passage de données dans les requêtes HTTP, notamment avec les méthodes GET avec paramètres, POST, PUT et DELETE et les récupérer en PHP.

À noter que vous pourriez appeler la même route (URL) avec des méthodes différentes pour réaliser des opérations différentes. Par exemple, vous pourriez appeler `/api/taches` en GET pour récupérer les tâches, en POST pour en ajouter une nouvelle, en PUT pour la modifier et en DELETE pour la supprimer. Du côté serveur, vous pouvez vérifier la méthode de la requête pour déterminer l'action à effectuer.

```
// Exemple de vérification de la méthode de la requête en PHP
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // Ajouter une nouvelle tâche
} elseif ($_SERVER['REQUEST_METHOD'] === 'PUT') {
    // Modifier une tâche existante
} elseif ($_SERVER['REQUEST_METHOD'] === 'DELETE') {
    // Supprimer une tâche
}
```

Objet de configuration de Fetch

La fonction `fetch` accepte un deuxième paramètre optionnel qui est un objet de configuration. Cet objet permet de spécifier des options supplémentaires pour la requête HTTP, telles que la méthode, les en-têtes, le corps de la requête, etc.

- `method` : La méthode HTTP à utiliser pour la requête (GET, POST, PUT, DELETE, etc.). Par défaut, c'est GET.
- `headers` : Un objet contenant les en-têtes HTTP à inclure dans la requête. Par exemple, pour spécifier le type de contenu.
 - `'Content-Type': 'application/json'` pour indiquer que le corps de la requête est au format JSON.
 - `'Authorization': 'Bearer token'` pour inclure un jeton d'authentification. Pour l'authentification.
 - `'Accept': 'application/json'` pour indiquer que le client attend une réponse au format JSON.
- `body` : Le corps de la requête, utilisé principalement avec les méthodes POST, PUT et DELETE. Il peut s'agir d'une chaîne de caractères, d'un objet FormData, d'un Blob, etc. Si vous envoyez des données JSON, vous devez les convertir en chaîne de caractères avec `JSON.stringify()`.

Exemple d'objet de configuration pour une requête POST avec des données JSON :

```
const config = {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
```

```
    },  
    body: JSON.stringify({ nom: "John", age: 30 }),  
  };
```

Envoi de données avec la méthode POST

Pour passer des données sensibles ou pour ajouter de la données à une base de données, on utilise la méthode POST. Les données sont envoyées dans le corps de la requête et non dans l'URL.

Habituellement, en PHP, les données envoyées par une requête POST sont accessibles via le tableau associatif `$_POST`. Cependant, lorsqu'elles sont envoyées par fetch, elles sont accessibles via la méthode `file_get_contents('php://input')` et doivent être décodées en JSON.

Fetch permet de passer un objet de configuration en deuxième paramètre de la fonction fetch. Cet objet contient les options de la requête, comme la méthode, les en-têtes et le corps de la requête.

C'est dans le corps de la requête que les données sont envoyées. Pour envoyer des données en POST, on doit spécifier le type de contenu dans les en-têtes de la requête avec `'Content-Type': 'application/json'` et convertir les données en JSON avec `JSON.stringify()`.

De plus, il faut indiquer la méthode de la requête dans l'objet de configuration avec `method: 'POST'`.

```
//Exemple de requête Fetch avec la méthode POST  
  
async function ajouterUneTache(tache) {  
  try {  
    const config = {  
      method: "POST",  
      headers: {  
        "Content-Type": "application/json",  
      },  
      body: JSON.stringify(tache),  
    };  
  
    const response = await fetch("https://api.example.com/ajouter-tache.php",  
config);  
    const data = await response.json();  
  
    //...Afficher la donnée ajoutée  
    // new Tache(data);  
  
    //Afficher la rétroaction de l'ajout  
    // new ToastModale("Tâche ajoutée", "success");  
  } catch (error) {  
    console.error(error);  
    //Afficher la rétroaction de l'erreur  
  }  
}
```

Du côté serveur, on récupère les données envoyées par la requête POST avec `file_get_contents('php://input')` et on les décode en JSON avec `json_decode()`. Ensuite, on peut utiliser ces données pour effectuer des opérations, comme insérer une nouvelle tâche dans une base de données.

```
//Exemple de récupération des données envoyées par une requête POST en PHP
$data = json_decode(file_get_contents('php://input'), true); //true pour retourner
un tableau associatif

//Mettre les paramètres de connexion dans un fichier séparé
require_once('config.php');

//Connexion
$pdoConnexion = new PDO("mysql:host=$host;dbname=$dbname;port=$port", $username,
$password);

// Requête à la base de données
// On prépare la requête avec des paramètres nommés pour éviter les injections SQL
$sql = "INSERT INTO taches (nom, date, description) VALUES (:nom, :date,
:description)";
$query = $pdoConnexion->prepare($sql);

// On exécute la requête en passant les valeurs des paramètres
$query->execute(array(
    'nom' => $data['nom'],
    'date' => $data['date'],
    'description' => $data['description']
));

//Retourne la réponse
$dernierId = $pdoConnexion->lastInsertId();
$reponse = array('id' => $dernierId, "message" => "Tâche ajoutée avec succès");

header("Content-Type: application/json");
status_header(200);
echo json_encode($reponse); //On retourne un objet JSON
```

Autres méthodes HTTP

Outre les méthodes GET et POST, il existe d'autres méthodes HTTP qui permettent de réaliser des opérations plus complexes sur les ressources. Cependant, ces méthodes ne sont pas disponibles sur les formulaires HTML et nécessitent l'utilisation de JavaScript pour les envoyer.

Les requêtes HTTP peuvent avoir la même url si elles ont des méthodes différentes.

- **PUT** : La méthode PUT est utilisée pour mettre à jour une ressource existante. Les données à mettre à jour sont envoyées dans le corps de la requête. En PHP, les données envoyées par une requête PUT sont accessibles via la méthode `file_get_contents('php://input')` et doivent être décodées en JSON. C'est comme la méthode POST, mais pour mettre à jour une ressource. Il faut passer les infos de la ressource à mettre à jour dans le corps de la requête et indiquer la méthode `put` pour mettre à jour la ressource.

- **DELETE** : La méthode DELETE est utilisée pour supprimer une ressource. Souvent, on utilise la même url que pour récupérer une ressource, mais avec la méthode DELETE. En PHP, les données envoyées par une requête DELETE sont accessibles via `$_GET`. Cependant, il faut passer l'id de la ressource à supprimer dans l'URL et la méthode `delete` pour supprimer la ressource. Il n'y a jamais rien dans le corps de la requête.

Modifier une tâche avec la méthode PUT

```
//Exemple de requête Fetch avec la méthode PUT

async function modifierUneTache(tache) {
  try {
    const config = {
      method: "PUT",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(tache),
    };

    const response = await fetch(`https://api.example.com/api/modifier-taches.php`, config);
    const data = await response.json();

    //...Afficher la donnée modifiée
    // new Tache(data);

    //Afficher la rétroaction de la modification
    // new ToastModale("Tâche modifiée", "success");
  } catch (error) {
    console.error(error);
    //Afficher la rétroaction de l'erreur
  }
}
```

```
//Exemple de récupération des données envoyées par une requête PUT en PHP
$data = json_decode(file_get_contents('php://input'), true); //true pour retourner un tableau associatif

//Connexion
$pdoConnexion = new PDO("mysql:host=$host;dbname=$dbname;port=$port", $username, $password);

// Requête à la base de données
// On prépare la requête avec des paramètres nommés pour éviter les injections SQL
$sql = "UPDATE taches SET nom = :nom, date = :date, description = :description WHERE id = :id";
$query = $pdoConnexion->prepare($sql);

// On exécute la requête en passant les valeurs des paramètres
```

```

$query->execute(array(
    'id' => $data['id'],
    'nom' => $data['nom'],
    'date' => $data['date'],
    'description' => $data['description']
));

//Retourne la réponse
$reponse = array('id' => $data['id'], "message" => "Tâche modifiée avec succès");

header("Content-Type: application/json");
status_header(200);
echo json_encode($reponse);//On retourne un objet JSON

```

Supprimer une tâche avec la méthode DELETE

```

//Exemple de requête Fetch avec la méthode DELETE

async function supprimerUneTache(id) {
    try {
        const config = {
            method: "DELETE",
        };

        const response = await fetch(`https://api.example.com/api/supprimer-taches.php?id=${id}`, config);
        const data = await response.json();

        //Afficher la rétroaction de la suppression
        // new ToastModale("Tâche supprimée", "success");
    } catch (error) {
        console.error(error);
        //Afficher la rétroaction de l'erreur
    }
}

```

```

//Exemple de récupération des données envoyées par une requête DELETE en PHP
$id = $_GET['id']; //Le paramètre est dans l'URL donc accessible via $_GET

//Connexion
require_once('config.php');

$pdoConnexion = new PDO("mysql:host=$host;dbname=$dbname;port=$port", $username, $password);

// Requête à la base de données
$sql = "DELETE FROM taches WHERE id = :id";
$query = $pdoConnexion->prepare($sql);
$query->execute(array('id' => $id));

```

```
//Retourne la réponse
$reponse = array('id' => $id, "message" => "Tâche supprimée avec succès");

header("Content-Type: application/json");
status_header(200);
echo json_encode($reponse); //On retourne un objet JSON
```